Equipotential Visualizer

Generated by Doxygen 1.9.8

1 Equipotential Simulator/Visualizer 1
1.1 Features . . . 1
1.2 Demonstrated Skills . . . 1
1.3 Installation & Build . . . 1
1.4 Usage Example . . . 1
1.5 Roadmap . . . 2
1.6 License . . . 2

2 Data Structure Index 2
2.1 Data Structures . . . 2

3 File Index 2
3.1 File List . . . 2

4 Data Structure Documentation 2
4.1 charge_t Struct Reference . . . 2
4.1.1 Detailed Description . . . 3
4.1.2 Field Documentation . . . 3

5 File Documentation 4
5.1 README.md File Reference . . . 4
5.2 src/charge.h File Reference . . . 4
5.2.1 Typedef Documentation . . . 5
5.2.2 Enumeration Type Documentation . . . 5
5.3 charge.h . . . 5
5.4 src/config.h File Reference . . . 6
5.4.1 Macro Definition Documentation . . . 7
5.5 config.h . . . 8
5.6 src/grid.h File Reference . . . 9
5.6.1 Variable Documentation . . . 9
5.7 grid.h . . . 10
5.8 src/main.c File Reference . . . 10
5.8.1 Function Documentation . . . 11
5.8.2 Variable Documentation . . . 12
5.9 src/render.c File Reference . . . 12
5.9.1 Macro Definition Documentation . . . 13
5.9.2 Function Documentation . . . 13
5.10 src/render.h File Reference . . . 15
5.10.1 Function Documentation . . . 16
5.11 render.h . . . 17
5.12 src/space_calc.c File Reference . . . 18
5.12.1 Function Documentation . . . 18
5.13 src/space_calc.h File Reference . . . 20
5.13.1 Function Documentation . . . 21

5.14 space_calc.h 22

Index 23

1 Equipotential Simulator/Visualizer

A high-resolution 2D electrostatic equipotential line simulator written in C.

1.1 Features

- User defined point charges
- High resolution rendering and output (2000x1000 by default)
- Contour drawing using LOG based mathematics
- Written in pure C (only external dependency being STB)

1.2 Demonstrated Skills

- **Low-level programming and mathematics**: This program demonstrates low level mathematics and memory management.
- **C Proficiency**: The lack of external dependecies and pure C showcases a proficiency in a highly-tedious and unforgiving programming language.
  - Modular design with header/source separation, structs, enums.
  - Grid-based field computation, distance calculations, avoiding singularities.
- **Image processing**: Integration with `stb_image_write.h` for PNG output to desired location.

1.3 Installation & Build

```
git clone https://github.com/Shrewd-Man/Equipotential-Visualizer.git
cd Equipotential-Visualizer

# Compile (from src directory or adjust paths)
gcc src/*.c -o equipotential -lm -I src
```

Run `./equipotential`

1.4 Usage Example

- 2 charges: +1 nC at (-2, 0), -1 nC at (2, 0) → classic dipole equipotential pattern.
- Supports up to 4 charges currently (easily extensible).

1.5 Roadmap

- **First Change**: addition of a grayscale heatmap-like render to outperform flat contours.
- Modular input to allow implementation in custom applications.
- Real-time data output formatted for custom rendering programs.
- 3D extention.

1.6 License

Use in compliance with the GPL-3.0 license

2 Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

charge_t
A single point charge in the simulated space 2

3 File Index

3.1 File List

Here is a list of all files with brief descriptions:

src/charge.h 4

src/config.h 6

src/grid.h 9

src/main.c 10

src/render.c 12

src/render.h 15

src/space_calc.c 18

src/space_calc.h 20

4 Data Structure Documentation

4.1 charge_t Struct Reference

A single point charge in the simulated space.

`#include <charge.h>`

Data Fields

- double space_x
- double space_y
- int point_x
- int point_y
- double magnitude
- sign_t sign

4.1.1 Detailed Description

A single point charge in the simulated space.

A charge is stored twice: once in physical coordinates (meters) and once as the grid point it lands on. The magnitude is always stored as a positive number; the polarity lives in the sign field.

4.1.2 Field Documentation

magnitude

```
double charge_t::magnitude
```

Absolute size of the charge in coulombs

point_x

```
int charge_t::point_x
```

X position as a grid point index (0 to NX - 1)

point_y

```
int charge_t::point_y
```

Y position as a grid point index (0 to NY - 1)

sign

```
sign_t charge_t::sign
```

Polarity: NEGATIVE (0) or POSITIVE (1)

space_x

```
double charge_t::space_x
```

X position in meters

space_y

```
double charge_t::space_y
```

Y position in meters

The documentation for this struct was generated from the following file:

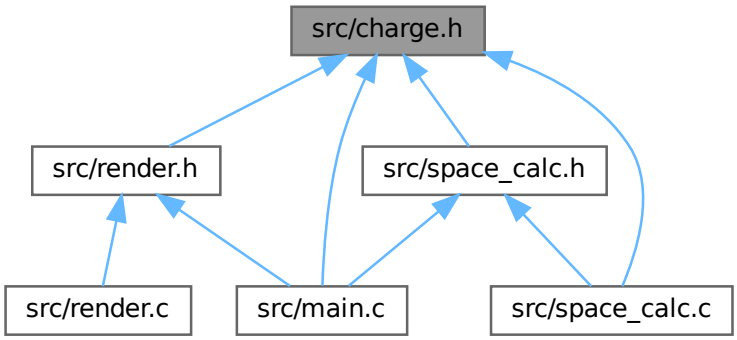
- src/charge.h

5 File Documentation

5.1 README.md File Reference

5.2 src/charge.h File Reference

This graph shows which files directly or indirectly include this file:



Data Structures

- struct charge_t

  A single point charge in the simulated space.

Typedefs

- typedef struct charge_t charge_t

  A single point charge in the simulated space.

Enumerations

- enum sign_t { NEGATIVE = 0 , POSITIVE = 1 }

  Polarity of a point charge.

5.2.1 Typedef Documentation

charge_t

```
typedef struct charge_t charge_t
```

A single point charge in the simulated space.

A charge is stored twice: once in physical coordinates (meters) and once as the grid point it lands on. The magnitude is always stored as a positive number; the polarity lives in the sign field.

5.2.2 Enumeration Type Documentation

sign_t

```
enum sign_t
```

Polarity of a point charge.

Enumerator

NEGATIVE	Negative charge
POSITIVE	Positive charge

5.3 charge.h

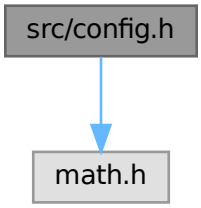
Go to the documentation of this file.

```
00001 // charge.h
00002
00003 #ifndef CHARGE_H
00004 #define CHARGE_H
00005
00006 // --- Sign Bit ---
00010 typedef enum {
00011     NEGATIVE = 0,
00012     POSITIVE = 1
00013 } sign_t;
00014
00015 // --- Charge Structure ---
00022 typedef struct charge_t {
00023     double space_x;
00024     double space_y;
00026     int point_x;
00027     int point_y;
00029     double magnitude;
00031     sign_t sign;
00032 } charge_t;
00033
00034 #endif //CHARGE_H
```

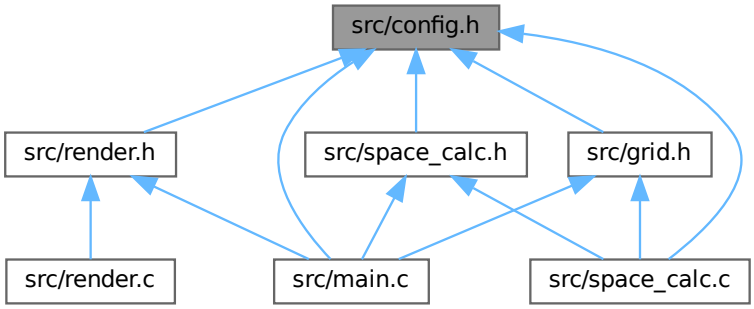
5.4 src/config.h File Reference

```
#include <math.h>
```

Include dependency graph for config.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define XMIN -10.0
- #define XMAX 10.0
- #define YMIN -5.0
- #define YMAX 5.0
- #define NX 2000
- #define NY 1000
- #define VOLT_INC 15
- #define CONTOUR_LINES 8
- #define CONTOUR_DECADES 2.0
- #define COULOMBS_CONSTANT (8.99e9)

5.4.1 Macro Definition Documentation

CONTOUR_DECADES

```
#define CONTOUR_DECADES 2.0
```

How far below its peak a sign's contours reach, in powers of ten

CONTOUR_LINES

```
#define CONTOUR_LINES 8
```

Contour lines drawn for each charge sign

COULOMBS_CONSTANT

```
#define COULOMBS_CONSTANT (8.99e9)
```

Coulomb's constant k, in N∗m^2/C^2

NX

```
#define NX 2000
```

Number of grid points along x (also the output image width in pixels)

NY

```
#define NY 1000
```

Number of grid points along y (also the output image height in pixels)

VOLT_INC

```
#define VOLT_INC 15
```

Voltage increment setting (currently unused)

XMAX

```
#define XMAX 10.0
```

Right edge of the simulated space, in meters

XMIN

```
#define XMIN -10.0
```

Left edge of the simulated space, in meters

YMAX

```
#define YMAX 5.0
```

Top edge of the simulated space, in meters

YMIN

```
#define YMIN -5.0
```

Bottom edge of the simulated space, in meters

5.5 config.h

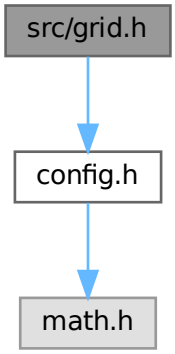
Go to the documentation of this file.

```
00001 // CONFIG.H
00002 #ifndef CONFIG_H
00003 #define CONFIG_H
00004
00005 // --- NECESSARY HEADERS ---
00006 #include <math.h>
00007
00008 // --- SIMULATION PARAMETERS ---
00009
00010     // Boundaries
00011 #define XMIN -10.0
00012 #define XMAX 10.0
00013 #define YMIN -5.0
00014 #define YMAX 5.0
00016     // Resolution
00017 #define NX 2000
00018 #define NY 1000
00020     // Voltage-Based Rendering increments
00021 #define VOLT_INC 15
00023     // Voltage-Based Rendering contours
00024 #define CONTOUR_LINES 8
00025 #define CONTOUR_DECADES 2.0
00027 // --- CONSTANTS ---
00028
00029     // Coulombs Constant
00030 #define COULOMBS_CONSTANT (8.99e9)
00033 #endif
```

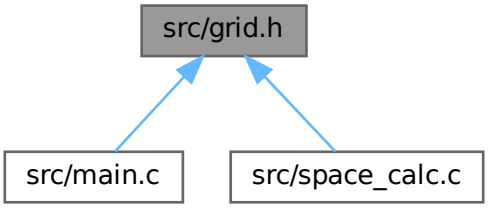
5.6 src/grid.h File Reference

```
#include "config.h"
```

Include dependency graph for grid.h:



This graph shows which files directly or indirectly include this file:



Variables

- double dx

  Spacing between neighbouring grid points along x, in meters (defined in main.c)
- double dy

  Spacing between neighbouring grid points along y, in meters (defined in main.c)

5.6.1 Variable Documentation

dx

```
double dx  [extern]
```

Spacing between neighbouring grid points along x, in meters (defined in main.c)

Spacing between neighbouring grid points along x, in meters (defined in main.c)

dy

```
double dy  [extern]
```

Spacing between neighbouring grid points along y, in meters (defined in main.c)

Spacing between neighbouring grid points along y, in meters (defined in main.c)

5.7 grid.h

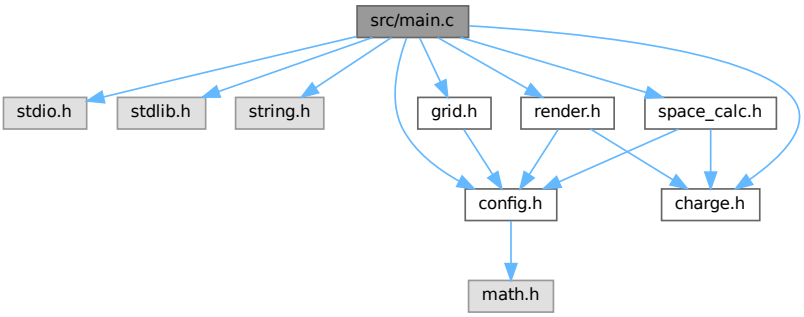
Go to the documentation of this file.

```
00001 // grid.h
00002 #ifndef GRID_H
00003 #define GRID_H
00004
00005 #include "config.h"
00006
00007 // --- UNIVERSAL GRID VALUES ---
00009 extern double dx;
00011 extern double dy;
00012
00013 #endif
```

5.8 src/main.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "config.h"
#include "charge.h"
#include "render.h"
#include "grid.h"
#include "space_calc.h"
```

Include dependency graph for main.c:



Functions

- int main (int argc, const char *argv[])

  Program entry point: reads the charges, computes the voltage field, and renders it to a PNG.

Variables

- double V [NX][NY]

  Voltage at every grid point, indexed V[x][y].
- double dx = (XMAX - XMIN) / (NX - 1)

  Grid spacing along x in meters, derived from XMIN, XMAX and NX.
- double dy = (YMAX - YMIN) / (NY - 1)

  Grid spacing along y in meters, derived from YMIN, YMAX and NY.

5.8.1 Function Documentation

main()

```
int main (
            int argc,
            const char * argv[] )
```

Program entry point: reads the charges, computes the voltage field, and renders it to a PNG.

The program runs in four steps:

- Asks for the number of charges (1 to 4), then each charge's position and size
- Computes the voltage at every grid point with setup_grid()
- Asks for an output file path (including the '.png' extension)
- Draws the equipotential contours with render_voltage_based() and writes the image

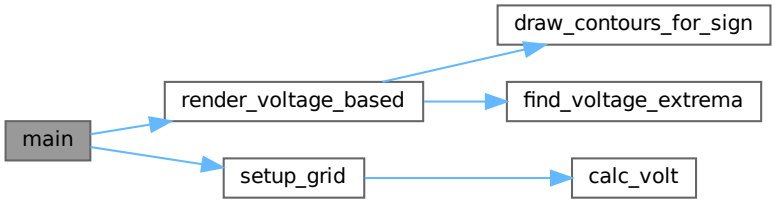
Parameters

argc	number of command-line arguments (unused)
argv	command-line arguments (unused)

Returns

0 on completion, also 0 if the requested charge count is outside 1 to 4

Here is the call graph for this function:



5.8.2 Variable Documentation

dx

```
double dx = (XMAX - XMIN) / (NX - 1)
```

Grid spacing along x in meters, derived from XMIN, XMAX and NX.

Spacing between neighbouring grid points along x, in meters (defined in main.c)

dy

```
double dy = (YMAX - YMIN) / (NY - 1)
```

Grid spacing along y in meters, derived from YMIN, YMAX and NY.

Spacing between neighbouring grid points along y, in meters (defined in main.c)

V

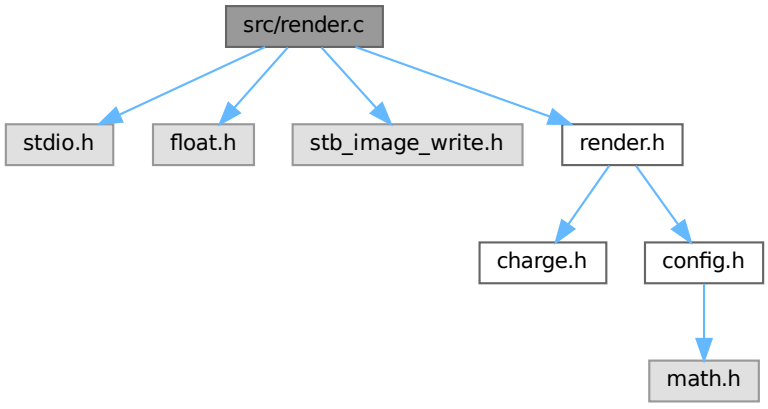
```
double V[NX][NY]
```

Voltage at every grid point, indexed V[x][y].

5.9 src/render.c File Reference

```
#include <stdio.h>
#include <float.h>
#include "stb_image_write.h"
#include "render.h"
```

Include dependency graph for render.c:



Macros

- #define STB_IMAGE_WRITE_IMPLEMENTATION

Functions

- double find_voltage_extrema (charge_t charge[ ], int charge_count, int type, double V[NX][NY])

  function finds the minimum and maximum voltage in the space, and returns either min or max
- void draw_contours_for_sign (double V[NX][NY], unsigned char img[NY][NX], int sign, double extremum, int num_lines)

  draws the equipotential contour lines for either the positive or the negative potentials
- void render_voltage_based (charge_t charge[ ], int charge_count, double V[NX][NY], const char ∗output_↩ path)

  Renders the voltage field as white equipotential contour lines on a black PNG.

5.9.1 Macro Definition Documentation

STB_IMAGE_WRITE_IMPLEMENTATION

```
#define STB_IMAGE_WRITE_IMPLEMENTATION
```

5.9.2 Function Documentation

draw_contours_for_sign()

```
void draw_contours_for_sign (
            double V[NX][NY],
            unsigned char img[NY][NX],
            int sign,
            double extremum,
            int num_lines )
```

draws the equipotential contour lines for either the positive or the negative potentials

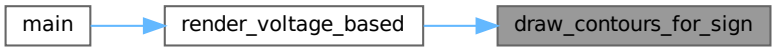
The contour levels are spaced logarithmically between the weakest and strongest potential of the requested sign (never more than CONTOUR_DECADES powers of ten below the strongest, so the lines stay in the part of the field that actually has structure).

Every point is assigned the number of contour levels it has passed, and a line is drawn wherever two neighbouring points have a different count. This keeps every line a constant, thin width no matter how steep or shallow the potential is, instead of painting a band around each level.

Parameters

V	the voltage coordinate array
img	the image to draw the lines into (255 for a line)
sign	+1 to draw the contours of the positive potentials, -1 for the negative potentials
extremum	the strongest voltage of this sign (taken outside the charges' exclusion zone)
num_lines	the number of contour lines to draw for this sign (2 or more)

Here is the caller graph for this function:



find_voltage_extrema()

```
double find_voltage_extrema (
            charge_t charge[],
            int charge_count,
            int type,
            double V[NX][NY] )
```

function finds the minimum and maximum voltage in the space, and returns either min or max

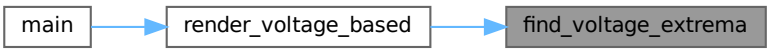
Parameters

charge	the array of charges (points within 30 grid cells of any charge are ignored)
charge_count	the total amount of charges
type	indicates return; 1 for max, 0 for min
V	array for voltage calcs

Returns

the maximum voltage if type is 1, otherwise the minimum voltage

Here is the caller graph for this function:



render_voltage_based()

```
void render_voltage_based (
            charge_t charge[],
            int charge_count,
            double V[NX][NY],
            const char * output_path )
```

Renders the voltage field as white equipotential contour lines on a black PNG.

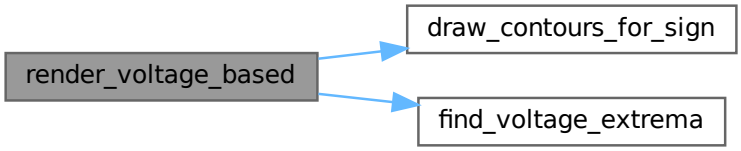
The function performs as follows:

- The function first determines the maximum and minimum voltage in the space
- Then, for the positive and the negative voltages separately, CONTOUR_LINES log-spaced voltage levels are chosen
- A line is drawn (white) wherever two neighbouring points (i, j) fall between different levels, everything else stays black

Parameters

charge	the array of charges (each is drawn as a single white dot)
charge_count	the total amounts of charges
V	the voltage coordinate array for which to render
output_path	the filepath (including the '.png' extension) to write the rendered image to

Here is the call graph for this function:



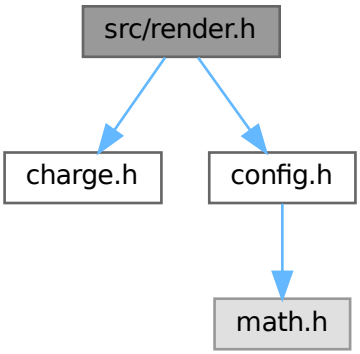
Here is the caller graph for this function:



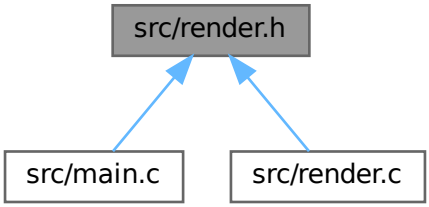
5.10 src/render.h File Reference

```
#include "charge.h"
#include "config.h"
```

Include dependency graph for render.h:



This graph shows which files directly or indirectly include this file:



Functions

- void render_voltage_based (charge_t charge[ ], int charge_count, double V[NX][NY], const char ∗output_↩ path)

  Renders the voltage field as white equipotential contour lines on a black PNG.

5.10.1 Function Documentation

render_voltage_based()

```
void render_voltage_based (
            charge_t charge[],
            int charge_count,
            double V[NX][NY],
            const char * output_path )
```

Renders the voltage field as white equipotential contour lines on a black PNG.

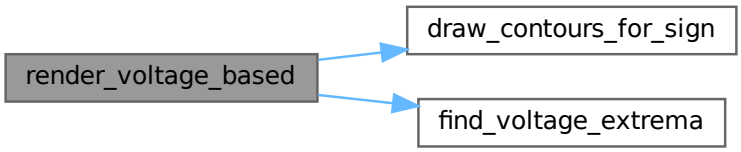
The function performs as follows:

- The function first determines the maximum and minimum voltage in the space
- Then, for the positive and the negative voltages separately, CONTOUR_LINES log-spaced voltage levels are chosen
- A line is drawn (white) wherever two neighbouring points (i, j) fall between different levels, everything else stays black

Parameters

charge	the array of charges (each is drawn as a single white dot)
charge_count	the total amounts of charges
V	the voltage coordinate array for which to render
output_path	the filepath (including the '.png' extension) to write the rendered image to

Here is the call graph for this function:



Here is the caller graph for this function:



5.11 render.h

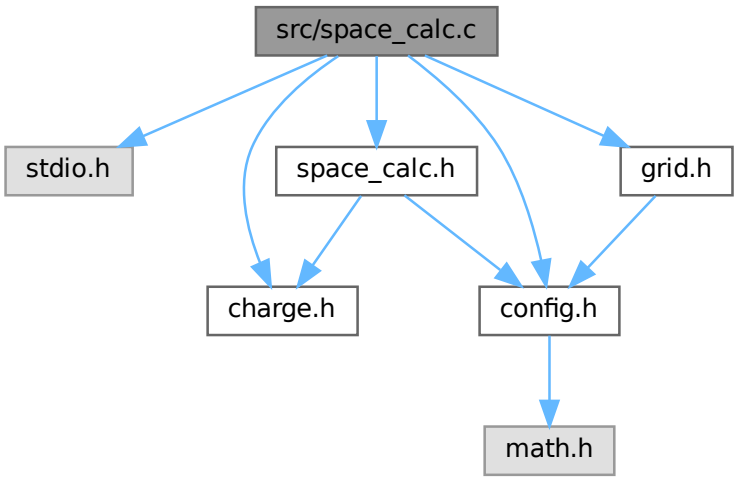
Go to the documentation of this file.

```
00001 // render.h
00002 #ifndef RENDER_H
00003 #define RENDER_H
00004
00005 #include "charge.h"
00006 #include "config.h"
00007
00008 // --- VOLTAGE BASED RENDERING ---
00009
00023 void render_voltage_based(charge_t charge[], int charge_count, double V[NX][NY], const char
      *output_path);
00024
00025 #endif
```

5.12 src/space_calc.c File Reference

```
#include <stdio.h>
#include "space_calc.h"
#include "charge.h"
#include "config.h"
#include "grid.h"
```

Include dependency graph for space_calc.c:



Functions

- double calc_volt (charge_t charge[ ], int charge_count, int x_grid, int y_grid)

  Function calculates the potential difference of any provided point.
- void setup_grid (charge_t charge[ ], int charge_count, double V[NX][NY])

  Function calculates the potential difference of every point in the matrix.

5.12.1 Function Documentation

calc_volt()

```
double calc_volt (
            charge_t charge[],
            int charge_count,
            int x_grid,
            int y_grid )
```

Function calculates the potential difference of any provided point.

The function uses the universal formula and currently computes that based on the points distance to the charge x and y

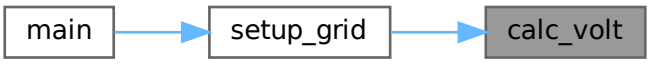
Parameters

charge	structure data with the charge values
charge_count	total amount of charges
x_grid	The x grid point of the calculation
y_grid	The y grid point of the calculation

Returns

Func returns the value of the potential difference at the given point

Here is the caller graph for this function:



setup_grid()

```
void setup_grid (
            charge_t charge[],
            int charge_count,
            double V[NX][NY] )
```

Function calculates the potential difference of every point in the matrix.

The function calls the calc_volt for every point in the space

Parameters

charge	the structure containing the data for the first charge
charge_count	the total amounts of charges
V	the voltage array to fill, one value per grid point

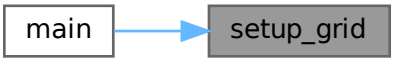
Note

This function is currently a temporary system that does not allow for live rendering

Here is the call graph for this function:



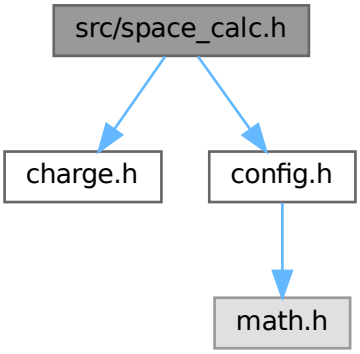
Here is the caller graph for this function:



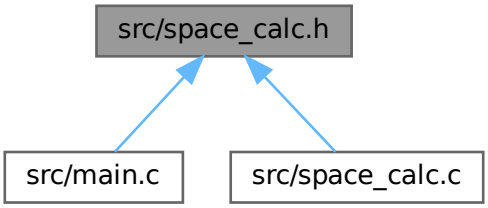
5.13 src/space_calc.h File Reference

```
#include "charge.h"
#include "config.h"
```

Include dependency graph for space_calc.h:



This graph shows which files directly or indirectly include this file:



Functions

- void setup_grid (charge_t charge[ ], int charge_count, double V[NX][NY])

  Function calculates the potential difference of every point in the matrix.

5.13.1 Function Documentation

setup_grid()

```
void setup_grid (
            charge_t charge[],
            int charge_count,
            double V[NX][NY] )
```

Function calculates the potential difference of every point in the matrix.

The function calls the calc_volt for every point in the space

Parameters

charge	the structure containing the data for the first charge
charge_count	the total amounts of charges
V	the voltage array to fill, one value per grid point

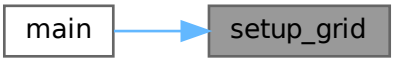
Note

This function is currently a temporary system that does not allow for live rendering

Here is the call graph for this function:



Here is the caller graph for this function:



5.14 space_calc.h

Go to the documentation of this file.

```
00001 // space_calc.h
00002 #ifndef SPACE_CALC_H
00003 #define SPACE_CALC_H
00004
00005 #include "charge.h"
00006 #include "config.h"
00007
00008 // --- VOLTAGE SPACE CALCULATION ALGORITHM ---
00009
00020 void setup_grid(charge_t charge[], int charge_count, double V[NX][NY]);
00021
00022
00023 #endif
```

Index

calc_volt
  space_calc.c, 18
charge.h
  charge_t, 5
  NEGATIVE, 5
  POSITIVE, 5
  sign_t, 5
charge_t, 2
  charge.h, 5
  magnitude, 3
  point_x, 3
  point_y, 3
  sign, 3
  space_x, 3
  space_y, 3
config.h
  CONTOUR_DECADES, 7
  CONTOUR_LINES, 7
  COULOMBS_CONSTANT, 7
  NX, 7
  NY, 7
  VOLT_INC, 7
  XMAX, 7
  XMIN, 7
  YMAX, 8
  YMIN, 8
CONTOUR_DECADES
  config.h, 7
CONTOUR_LINES
  config.h, 7
COULOMBS_CONSTANT
  config.h, 7

draw_contours_for_sign
  render.c, 13
dx
  grid.h, 9
  main.c, 12
dy
  grid.h, 9
  main.c, 12

Equipotential Simulator/Visualizer, 1

find_voltage_extrema
  render.c, 14

grid.h
  dx, 9
  dy, 9

magnitude
  charge_t, 3
main
  main.c, 11
main.c
  dx, 12
  dy, 12
  main, 11
  V, 12

NEGATIVE
  charge.h, 5
NX
  config.h, 7
NY
  config.h, 7

point_x
  charge_t, 3
point_y
  charge_t, 3
POSITIVE
  charge.h, 5

README.md, 4
render.c
  draw_contours_for_sign, 13
  find_voltage_extrema, 14
  render_voltage_based, 14
  STB_IMAGE_WRITE_IMPLEMENTATION, 13
render.h
  render_voltage_based, 16
render_voltage_based
  render.c, 14
  render.h, 16

setup_grid
  space_calc.c, 19
  space_calc.h, 21
sign
  charge_t, 3
sign_t
  charge.h, 5
space_calc.c
  calc_volt, 18
  setup_grid, 19
space_calc.h
  setup_grid, 21
space_x
  charge_t, 3
space_y
  charge_t, 3
src/charge.h, 4, 5
src/config.h, 6, 8
src/grid.h, 9, 10
src/main.c, 10
src/render.c, 12
src/render.h, 15, 17
src/space_calc.c, 18
src/space_calc.h, 20, 22
STB_IMAGE_WRITE_IMPLEMENTATION

render.c, 13

V
main.c, 12
VOLT_INC
config.h, 7

XMAX
config.h, 7
XMIN
config.h, 7

YMAX
config.h, 8
YMIN
config.h, 8